\newpage

Overview

strontium.os is a portfolio operating system built by Brian Ndege. It runs as both a web app (Vercel) and a native desktop application (Electron on macOS, Windows, Linux). It simulates a full desktop environment — with real apps, a live backend, and a working terminal — built on Next.js 15, React 19, Framer Motion, and TypeScript.

The OS is connected to the **AxiraNews backend** (Fastify + PostgreSQL + Redis) at `http://10.0.0.75:4000`, which serves live news, health data, and ingestion status across several apps.

The Desktop

Window Management

Every app opens in a floating, draggable window. Windows can be:

Action	How
Move	Click and drag the title bar
Resize	Drag any edge or corner
Minimize	Click the yellow – button in the title bar
Maximize / restore	Click the green + button
Close	Click the red × button
Bring to front	Click anywhere on the window

Multiple windows can be open at the same time. Minimized windows appear in the Dock with a dot indicator.

The Dock

The Dock sits at the bottom of the screen. Hovering over it magnifies icons (if magnification is enabled in Settings).

- **Single click** — opens the app
- **Click on an open app** — brings it to front, or unminimizes it
- **Dot below icon** — app is currently open
- **Yellow dot** — app is minimized

Dock Apps

Icon	App	Description
Terminal	**Terminal**	Shell emulator with real PTY in Electron
Person	**About**	Profile and bio

Newspaper	**AxiraNews**	Live news from the AxiraNews backend
Folder	**Projects**	Project showcase
Activity	**System Monitor**	Real CPU/RAM/disk + backend health
Database	**Skills.db**	Technology skill browser
Mail	**Contact**	Contact information
File	**Resume**	Full résumé
Book	**Notebook**	JupyterLab integration
Zap	**Simulation**	Data flow, network graph, and threat map
Onion	**Tor Browser**	Onion-routed browsing simulation
Globe	**Clearnet Browser**	Admin-gated unrestricted browser
Gear	**Settings**	Personalize the OS

Wallpaper

The desktop background renders a live animated circuit-board pattern. The style can be changed in Settings:

- **Grid** — circuit lines with node glow
- **Dots** — subtle dot matrix
- **Noise** — grain texture
- **None** — solid dark background

Apps

Terminal

The Terminal is the primary interface for interacting with strontium.os. In Electron, it spawns a **real PTY shell** (zsh/bash) using `node-pty`. In the browser, it runs a built-in mock shell.

Real PTY (Electron only)

When running in Electron, the terminal connects to your actual shell. It supports:

- Full color output (`xterm-256color`)
- Tab completion, history (arrow keys), all shell shortcuts
- Running any real command (`ls` , `git` , `python` , etc.)
- Resizes dynamically with the window

Mock Shell (browser / PTY fallback)

If running in a browser or if PTY fails, a feature-rich simulated shell is used.

Built-in Commands

Command	Description
`help`	Show all available commands
`whoami`	About Brian Ndege
`neofetch`	System profile snapshot (ASCII art)
`ls`	List virtual directory
`cat <file>`	Read `resume.md`, `about.txt`, or `contact.txt`
`echo <text>`	Print text
`pwd`	Print working directory (`/home/brian`)
`date`	Current date and time
`uname`	Kernel info
`ps`	Running processes
`history`	Command history
`clear` / `cls`	Clear the terminal
`ping <host>`	Simulated ping

App Launch Commands

Command	Opens
`open <app>`	Open any app by ID
`about`	About Brian
`axira`	AxiraNews
`projects`	Projects
`skills`	Skills.db
`contact`	Contact
`resume`	Resume
`notebook`	JupyterLab
`simulation`	Simulation Engine
`tor`	Tor Browser
`clearnet` / `browser`	Clearnet Browser
`sudo clearnet`	Open Clearnet with root flavor text
`settings`	Settings

AxiraNews Commands

Command	Description
`axira status`	Check live backend health (API, Postgres, Redis)
`axira news`	Fetch latest 5 headlines from backend
`axira search <query>`	Search articles by keyword
`axira ingest`	Trigger news ingestion pipeline (admin)

Keyboard Shortcuts

Shortcut	Action
`↑ / ↓`	Navigate command history
`Ctrl+L`	Clear terminal
`Ctrl+C`	Cancel current input
`Enter`	Execute command

Terminal Themes

Configurable in **Settings → Terminal**:

Theme	Description
Void	Dark background, warm gold cursor (default)
Matrix	Green-on-black, classic hacker aesthetic
Amber	Orange-gold phosphor monitor look
Ice	Deep blue with cyan accent

Terminal Settings

Setting	Options
Font size	11 – 20 px
Theme	Void, Matrix, Amber, Ice
Cursor style	Block, Bar, Underline
Scrollback lines	1000 – 20000

AxiraNews

The AxiraNews app connects to the live AxiraNews backend and displays real-time global news.

Features

- **Live feed** — articles fetched from `/v1/news` on the backend
- **Breaking ticker** — cycles through top headlines at the top of the window
- **Category tabs** — filter by Cybersecurity, Finance, Science, Technology, Politics, Health, Business, Global
- **Search** — full-text search via `/v1/news/search`
- **Article expand** — click any article to read the summary
- **Urgent badge** — red URGENT label on high-priority articles
- **External link** — opens the original source in your browser
- **Online/Offline indicator** — shows live status and API base URL

Offline Fallback

If the backend is unreachable, the app displays 5 cached demo articles and shows an **OFFLINE · cached** indicator. All functionality degrades gracefully.

Refresh

Click the rotating arrow button in the top-right to manually refresh all articles.

System Monitor

Displays real-time system metrics and backend health.

Resource Gauges

Four gauges show current usage as animated bar charts:

Gauge	Source
CPU	Real value via Electron IPC (`systeminformation`)
RAM	Real value via Electron IPC
Disk	Real value via Electron IPC (mount `/`)
Network	Calculated from `netRx + netTx` bytes/sec

In the browser (non-Electron), all gauges use animated mock values with realistic drift.

Click any gauge to highlight it.

Global Visitors

Shows a simulated 24-hour visitor count by country with animated bar charts. Click any country to expand session details (sessions, average duration, bounce rate).

Services Panel

Shows live health for all backend services:

Service	How it's checked
AxiraNews API	Real HTTP health check every 30 seconds
PostgreSQL	Real DB ping via backend `/health`

Redis	Real Redis ping via backend `/health`
Cloudflare CDN	Static (always green)
strontium.os	Static (always green — you're using it)

Status indicators:

- Green dot — online
- Yellow pulsing — checking or degraded
- Red dot — offline

Click any service to ping it. The latency (ms) is shown in real time.

Tor Browser

The Tor Browser simulates onion-routed browsing. It **auto-builds a relay circuit on open** and allows navigation to any URL via iframe (in Electron, where X-Frame-Options headers are stripped).

Circuit Panel

Shows the relay chain: **You → Guard → Middle → Exit → Destination**

Each node displays:

- Node name and country flag
- Fake IP address
- Encryption status

The circuit is drawn as an animated canvas path connecting all nodes.

Navigation

1. Wait for the circuit to build (a few seconds on open)
2. Type a URL in the address bar — bare hostnames like `google` are expanded to `https://google.com`
3. Press `Enter` or click the arrow to navigate
4. The site loads in the iframe below

Controls

Button	Action
New Circuit	Rebuild the relay path with new nodes
Circuit	Toggle the circuit panel sidebar

Log Console

A scrollable log at the bottom shows circuit build events, connection steps, and encryption handshake simulation.

> ***Note:*** *The Tor routing is a simulation — traffic does not actually route through the Tor network. Real URLs load via iframe directly.*

Clearnet Browser

The Clearnet Browser is an unrestricted iframe browser **locked behind an admin password**.

Unlocking

1. The app opens to a lock screen with a security warning
2. Enter the admin password when prompted
3. After 3 wrong attempts, a hint is shown: *"the OS name"*
4. Password: **strontium**

Using the Browser

Once unlocked:

- Type any URL in the address bar and press `Enter`
- A yellow **CLEARNET · unencrypted** strip is always visible as a reminder
- Click **lock** (top right) to re-lock and return to the password screen

> *Traffic in the Clearnet browser is unencrypted and directly traceable — unlike the Tor browser.*

About

Displays Brian Ndege's personal profile, background, current focus, and a brief bio. Includes profile image, role description, and links.

Projects

Showcases Brian's main projects with descriptions, technology stacks, and links:

- **AxiraNews** — AI-powered global news intelligence platform
- **strontium.os** — This portfolio OS
- **Axira AV** — Custom antivirus engine
- **OSINT Suite** — IP/domain/CVE intelligence tools

Skills.db

A browsable database of Brian's technical skills, organized by category:

- Languages (TypeScript, Python, Swift, Go, SQL)
- Frontend (Next.js, React, SwiftUI, Tailwind)
- Backend (Fastify, FastAPI, Node.js)
- Database (PostgreSQL, Redis, Prisma, SwiftData)
- AI/ML (Recommendation Systems, NLP, Local LLMs)
- Security (OSINT, JWT/OAuth, Antivirus Systems)
- Cloud (Railway, Vercel, Cloudflare, Azure)

Resume

A formatted, readable version of Brian's résumé, covering education, experience, projects, and skills. Rendered inside the OS window.

Contact

Displays contact information:

Channel	Address
Email	brian@strontiumnews.com
GitHub	github.com/briandege
LinkedIn	linkedin.com/in/briandege
Web	axiranews.com

Notebook

Launches **JupyterLab** in an embedded iframe. Used for data analysis, prototyping, and live Python notebooks. Requires a running JupyterLab server locally.

Simulation Engine

An interactive visualization suite with three panels:

1. Network Graph

An animated force-directed graph showing interconnected network nodes (Routers, Servers, Endpoints, Firewalls, IoT devices). Nodes pulse and edges animate to represent live data flow. Click nodes to inspect them.

2. Data Flow

Visualizes a data pipeline with animated particles flowing through stages:

`Ingest → Parse → Enrich → Score → Publish`

Each stage shows throughput rate and processing time. Particles are color-coded by event type.

3. Monte Carlo Simulation

Runs a live Monte Carlo simulation plotting probability paths over time. Multiple scenario lines are rendered with a confidence band, showing convergence behavior in stochastic systems.

4. Threat Map

A grid-based threat intelligence map showing attack vectors across services (API Gateway, Auth Service, Database, File Storage, Email Service). Color-coded threat levels update in real time.

Settings

Customize the strontium.os experience. All settings persist across sessions via `localStorage`.

Appearance

Setting	Options
Accent color	Gold, Cyan, Purple, Green, Red
Wallpaper	Grid, Dots, Noise, None
Dock magnification	On / Off
Animation speed	Fast, Normal, Slow
Reduce motion	On / Off

Terminal

Setting	Options
Theme	Void, Matrix, Amber, Ice
Font size	11 – 20 px (slider)
Cursor style	Block, Bar, Underline
Scrollback	1000 – 20000 lines

Startup Apps

Choose which apps open automatically when strontium.os launches. Click any app icon to toggle it on/off as a startup app.

Default startup apps: **Terminal, AxiraNews**

Running strontium.os

As a Web App

The web version runs on Vercel and is accessible from any browser. Terminal uses the mock shell. System metrics use animated mock values.

```
https://brian-os.vercel.app
```

In Electron (Desktop)

Running in Electron enables additional features:

- **Real PTY terminal** — actual shell (zsh/bash) via `node-pty`
- **Real system metrics** — CPU, RAM, disk, network via `systeminformation`
- **Iframe browsing** — X-Frame-Options headers are stripped, so websites load in Tor/Clearnet browsers

Starting in Development

```
# Start Next.js dev server + Electron together
npm run electron:dev

# Or start them separately:
npm run dev          # Next.js on port 3002
npm run electron     # Electron (connects to port 3002)
```

Building for Distribution

```
npm run dist:mac     # macOS .dmg (arm64 + x64)
npm run dist:win     # Windows .exe (NSIS installer)
npm run dist:linux   # Linux AppImage + .deb + .rpm
npm run dist:all     # All platforms
```

Environment Variables

Variable	Default	Description
`NEXT_PUBLIC_AXIRA_API_URL`	`http://10.0.0.75:4000`	AxiraNews backend URL
`ELECTRON_PORT`	`3002`	Dev server port for Electron to connect to
`NODE_ENV`	`development`	Set to `production` for builds

Backend (AxiraNews API)

strontium.os connects to the AxiraNews backend for live data.

Starting the Backend

```
cd AXIRA/Backend
npx tsx src/server.ts
```

Requires PostgreSQL running with database `axira` and user `postgres`.

```
# Start PostgreSQL (macOS via Homebrew)
brew services start postgresql@14

# Run database migrations
npx prisma migrate deploy --schema=src/db/prisma/schema.prisma
```

Health Check

```
GET http://localhost:4000/health
```

Returns:

```
{ "status": "ok", "db": "connected", "ts": "..." }
```

Key Endpoints Used by the OS

Endpoint	Used By
`GET /health`	System Monitor, Terminal (`axira status`)
`GET /v1/news`	AxiraNews app
`GET /v1/news/top-headlines`	AxiraNews ticker
`GET /v1/news/search?q=`	AxiraNews search, Terminal
`POST /v1/ingest`	Terminal (`axira ingest`)

Architecture

```
strontium.os
├── Next.js 15 (App Router)     — frontend framework
├── React 19                    — UI components
├── Framer Motion               — window animations, gauges
├── Zustand                     — window state, settings state
├── TypeScript                  — type safety
├── Tailwind CSS v4             — utility styles
└── Electron 41                 — desktop wrapper
    ├── node-pty                — real PTY terminal
    └── systeminformation       — real system metrics

AxiraNews Backend
├── Fastify                     — HTTP server
├── TypeScript                  — type safety
├── Prisma + PostgreSQL         — database ORM
├── Redis                       — caching
└── Railway / local             — deployment
```

Data Flow

```
User action in OS
  → React component
    → axiraClient.ts (fetch with fallback)
      → AxiraNews API (Fastify)
        → PostgreSQL / Redis
          → Response → component state → UI update
```

Keyboard Reference

Window Controls

Key / Action	Behavior
Click title bar + drag	Move window
Drag window edge	Resize
× button	Close window
– button	Minimize
+ button	Maximize / restore

Terminal (Mock Shell)

Key	Action
`Enter`	Run command
`↑`	Previous command
`↓`	Next command
`Ctrl+C`	Cancel input
`Ctrl+L`	Clear screen

Terminal (Real PTY — Electron)

All standard shell shortcuts work: `Tab` completion, `Ctrl+A/E`, `Ctrl+R` history search, etc.

Troubleshooting

The desktop looks blank / no apps open

Settings or window state may be corrupted. Open the browser console and run:

```
localStorage.clear(); location.reload();
```

Terminal shows "PTY failed to spawn"

This means `node-pty` is not available in the Electron build. The terminal falls back to the mock shell automatically. To fix:

```
npm install node-pty
npm rebuild node-pty
```

AxiraNews shows "OFFLINE · cached"

The backend is not reachable. Check:

1. Is the backend running? (`curl http://localhost:4000/health`)
2. Is PostgreSQL running? (`brew services list | grep postgres`)
3. Is `NEXT_PUBLIC_AXIRA_API_URL` set correctly in `.env.local` ?

Websites not loading in Tor / Clearnet browser

This only works in Electron (where X-Frame-Options headers are stripped). In the browser version, most sites will block iframing. This is a security restriction of the web platform.

System metrics showing 0% or animated values

System metrics require Electron + the `systeminformation` package. In the browser, animated mock values are shown instead. This is expected behavior.

About the Author

Brian Ndege is a software engineer, AI systems builder, and cybersecurity specialist. He builds intelligence infrastructure — from the data layer up.

Email	brian@strontiumnews.com
GitHub	github.com/briandege
LinkedIn	linkedin.com/in/briandege
Web	axiranews.com

Stack: TypeScript · Python · Swift · Go · SQL · Next.js · React · SwiftUI · FastAPI · PostgreSQL · Redis · Prisma · Railway · Vercel · Cloudflare · Azure

---

strontium.os v0.1.0 — built with Next.js 15, React 19, Framer Motion, Electron 41